Lab 2.a demo walkthrough — the ADR, step by step

The instructor's demo ("Six steps — follow along on screen"), written down. Use it to **replay the demo at your own pace**, to catch up if you missed the session, or to check your own ADR against a worked example. The instructor uses the same document as the demo script — there is no secret version.

Sample Claude replies below are **abridged and illustrative** — yours will differ in wording. What must match is the *shape* of each step: what you ask, what you check, what you decide.

Before you start: your own copy, branch `feat/governance`, `npm test` green.

Step 1 — Create the file from the template

```
cp docs/adr/template.md docs/adr/0002-repository-only-store-access.md
```

Naming rules: **next number** in `docs/adr/` (0001 exists, so 0002), then a **kebab-case title** that names the decision, not the topic (`repository-only-store-access`, not `database-stuff`). The number is an identity — never reused, even if the ADR is later superseded.

Fill the metadata block now, so it isn't forgotten:

```
- **Status:** Accepted
- **Date:** 2026-08-13
- **Deciders:** <your name>
```

> **Why "Accepted" for a decision someone else made?** Recording is not deciding. This is a *retroactive* ADR — it captures the *why* of a decision that already governs the code, before that why evaporates. The date is when it was recorded, not when it was decided. In a real team, Deciders names the people who made the call — which is rarely the person writing it up.

Step 2 — Prompt 1: gather evidence

Ask Claude to **read, not conclude**:

```
Read @server/src/repositories/ and @server/src/services/todos.service.ts.
What boundary do these files enforce, and what would break if it were
violated? Don't propose changes.
```

Example reply (abridged):

```
The four repository modules are the only files under server/src that import
../db/store (readDb/writeDb). Services import repositories, never the store.
The boundary this enforces: persistence details are confined to one layer.
If a service imported the store directly, business logic would couple to the
JSON-file storage format — swapping storage, or faking it in tests, would
then require touching business code.
```

Notice what the prompt did *not* ask: whether the boundary is a good idea. Evidence first.

Step 3 — Check the answer against the code

Don't trust it — check it. Ten seconds:

```
grep -rn "db/store" server/src --include='*.ts' | grep -v test
```

You'll find the four repositories — **and one more**: `server/src/index.ts` imports `ensureDb` from the store at boot. Claude's tidy summary ("only repositories import the store") missed a real, legitimate exception: the entrypoint has to initialize the database before anything else runs.

This is the whole reason step 3 exists. Correct the record in conversation:

```
grep also shows server/src/index.ts importing ensureDb from the store.
That's the boot path, not data access. Note it as a named exception
when we draft.
```

A wrong Context poisons everything after it — and an ADR that overstates its rule ("*only* repositories, no exceptions") is the kind of document people learn to distrust the first time they read the code.

Step 4 — Prompt 2: draft, with the Decision left open

```
Using @docs/adr/template.md, draft an ADR for that boundary. Fill Context and
Consequences from what you just read, including the index.ts exception.
Leave Decision as a question for me to answer.
```

Example reply (abridged):

```
## Context
TaskBoard persists to a JSON file behind server/src/db/store.ts. All four
repository modules import readDb/writeDb; services call repositories;
routes call services. The single exception is server/src/index.ts, which
calls ensureDb() once at startup. Without a stated rule, nothing stops a
service importing the store directly, coupling business logic to the
storage format.

## Decision
[Left for you: what exactly is the rule, and does the boot-time exception
get named in it?]

## Consequences
- Good: persistence is swappable without touching business logic
- Good: tests can fake the store at one seam (makeTestDb)
- Bad: simple reads pay an extra hop through a repository
```

The draft is raw material. The bracketed question is the point — Claude gathered, you decide.

Step 5 — Edit the Context; write the Decision yourself

Read Claude's Context **as someone who disagrees**. Tighten anything vague, delete anything you can't verify. Then write the Decision — one sentence, present tense, active voice, and *yours*:

```
## Decision

Only modules under `server/src/repositories/` may import `server/src/db/store.ts`;
the single exception is the boot-time `ensureDb()` call in `server/src/index.ts`.
```

Two tests for a good Decision sentence:

- **No verbs of choosing.** "We decided to…" belongs nowhere — state the rule as fact.
- **You could defend it.** If the sentence came out of Claude and you couldn't argue for it against a sceptic, rewrite it in your own words until you could.

Step 6 — Add one honest cost, then stage it

An ADR with only upsides is marketing. The Consequences section needs at least one entry a real person could complain about on a real day:

```
## Consequences

- Good: persistence is swappable without touching business logic
- Good: tests fake the store at one seam (`makeTestDb()`)
- Bad: simple reads pay an extra hop — a one-line query still goes
  route → service → repository
- Bad: every new resource needs one more file (its repository)
```

Weak cost (really an upside wearing a frown): *"Bad: developers must learn the pattern."* Honest cost: the extra hop, the extra file. Test: would a sceptic nod?

Then stage and commit on your branch:

```
git add docs/adr/0002-repository-only-store-access.md
git commit -m "docs: record repository-only store access decision (ADR-0002)"
```

The finished ADR, in full

```
# ADR-0002: Repository-only store access

- **Status:** Accepted
- **Date:** 2026-08-13
- **Deciders:** <your name>

## Context

TaskBoard persists to a JSON file behind `server/src/db/store.ts`. All four
repository modules import `readDb`/`writeDb`; services call repositories;
routes call services. The single exception is `server/src/index.ts`, which
calls `ensureDb()` once at startup — boot, not data access. Without a stated
rule, nothing stops a service importing the store directly and coupling
business logic to the storage format.

## Decision

Only modules under `server/src/repositories/` may import
`server/src/db/store.ts`; the single exception is the boot-time `ensureDb()`
call in `server/src/index.ts`.

## Consequences

- Good: persistence is swappable without touching business logic
- Good: tests fake the store at one seam (`makeTestDb()`)
- Bad: simple reads pay an extra hop — route → service → repository
- Bad: every new resource needs one more file (its repository)
- Follow-up: nothing _checks_ this rule today — enforcement is Lab 2.b
```

Common wobbles

Wobble	The move
Claude’s evidence is wrong or incomplete	Correct it in conversation (step 3) and re-ask — bad evidence makes a bad Context
Context contains “we decided…”	Verbs of choosing move down to the Decision; Context describes forces
The Decision is a paragraph	One sentence. If it needs two, the second is probably a consequence or an exception
Every consequence is an upside	Add the cost a sceptic would name — the extra hop is sitting right there
The ADR sprawls past a page	You’re writing documentation, not a decision. Cut Context to 3–6 sentences
“Mine came out nearly identical to the demo’s”	Expected on a shared subject — the checkpoint checks the metadata block, the three sections, and a real cost, not originality